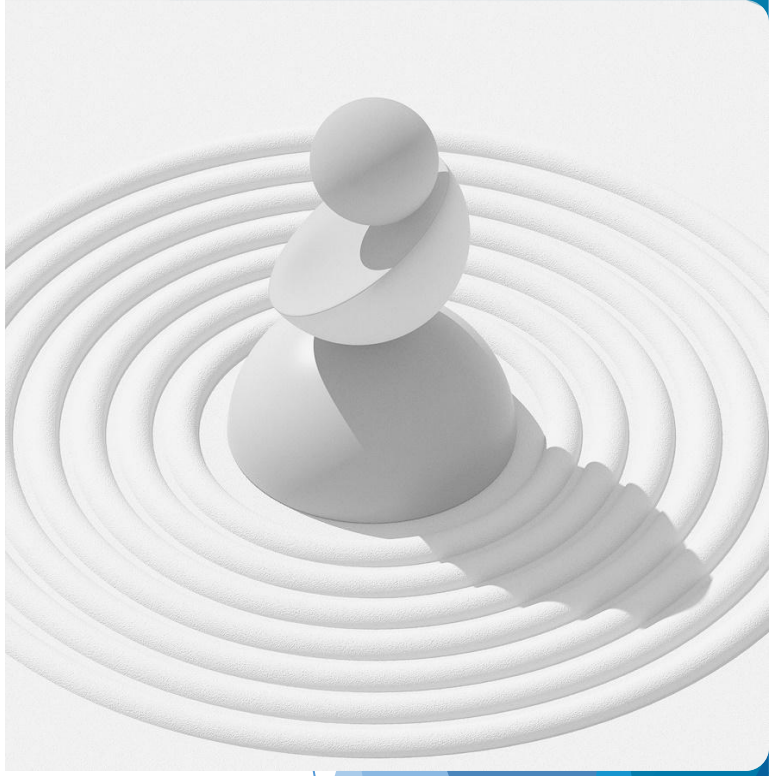
Presented by Nirvana Popescu

Lesson 1:Configuration Management

Module: Advanced Software Engineering

Outline

- What is Configuration Management?
- Why Configuration Management Matters
- Example of Bad CM
- Configuration Management and Safety Standards
- Configuration Identification Strategy
- Baselines and Reproducibility
- Versioning Strategy
- Branching Strategy
- Git as a Configuration Management Tool
- GitHub Adds the Control Workflow
- Configuration Management for a Scade One Project
- Traceability Through Configuration Management



What is Configuration Management?

- Configuration Management (CM) is the systematic process of identifying, organizing, tracking, and maintaining software artifacts throughout the Software Development Life Cycle (SDLC).
- Why is CM important?
  - Ensures that only approved and verified versions are used.
  - Maintains traceability and reproducibility of software.
  - Supports compliance with safety standards.
- How is CM implemented?
  - Tools such as Git and GitHub help manage changes to requirements, Scade One models, source code, tests, and documentation.
- **Main Learning Objective:**
  Understand how standard configuration management tools can be used to control the evolution of a Scade One project while supporting safety, traceability, and reproducibility.

Why Configuration Management Matters

- In critical systems, software continuously evolves:
  - Requirements are modified
  - Models and source code change
  - Tests are added or updated
  - Defects are corrected
  - Safety analyses evolve
  - New software versions are released
- Without rigorous configuration management:

Change → Inconsistency → Loss of Traceability → Uncontrolled Risk

- Configuration Management ensures that we can answer:

What changed, who changed it, why was it changed, and which system version contains the change?

Example of Bad CM



- What happened?
  - An Airbus A400M crashed near Seville, Spain, in 2015.
  - Engine control software was improperly configured during installation.
  - The problem involved software configuration data, rather than an identified error in the control algorithm.
- What went wrong?
  - Incorrect configuration data affected the operation of the engine control units, contributing to the loss of engine power.
- Key takeaway
  - Even correctly designed software can lead to catastrophic failure if it is improperly configured or installed.
  - Configuration Management is essential for ensuring that the correct software and configuration data are delivered to the correct system.

Configuration Management and Safety Standards

- Safety-critical development standards require controlled management of software artifacts and changes.
- Examples include:
  - **DO-178C** — airborne software
  - **ISO 26262** — automotive functional safety
  - **IEC 62304** — medical device software
  - **EN 50716** — railway software
- Typical expectations include:
- Unique identification of configuration items
  - Version control
  - Change control
  - Baseline management
  - Configuration status accounting
  - Reproducible builds/releases
  - Controlled access and modification
  - Traceability of changes

CM provides evidence that the assessed and tested software corresponds to the software that is actually released.

Configuration Identification Strategy

Objective: define configuration items

- A critical software project contains more than source code.
- Configuration Items (CIs) may include:
  - Requirements
  - Scade One models
  - Swan source/model files
  - Interfaces and data definitions
  - Generated code
  - Manually developed source code
  - Test cases and test procedures
  - Test results
  - Configuration files
  - Build scripts
  - Safety-related documentation
  - Release documentation
- A configuration therefore represents a consistent set of artifacts describing a particular system version.

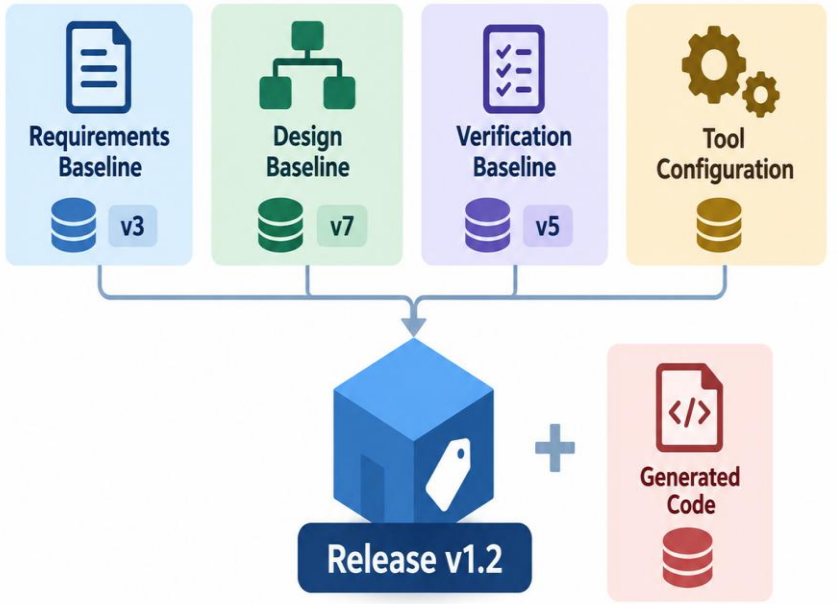
If an artifact can influence the behavior, verification, or safety argument of the software, its version should be controlled.

Baselines and Reproducibility

Objective: create stable starting points

- A **baseline** is an identified and approved configuration used as a reference point.
- Examples:
  - Requirements Baseline
  - Design Baseline
  - Verification Baseline
  - Release Baseline
- For example:
  - **Release v1.2** may identify:

  Requirements v3 + SCADE Model v7 + Tests v5 + Tool Configuration + Generated Code
- A good CM process should make it possible to retrieve the baseline and reproduce the corresponding software.
- **Reproducibility principle:**
  - Same controlled inputs + same controlled tools/configuration → reproducible software artifact
- This is particularly important for verification, audits, certification, maintenance, and incident investigation.

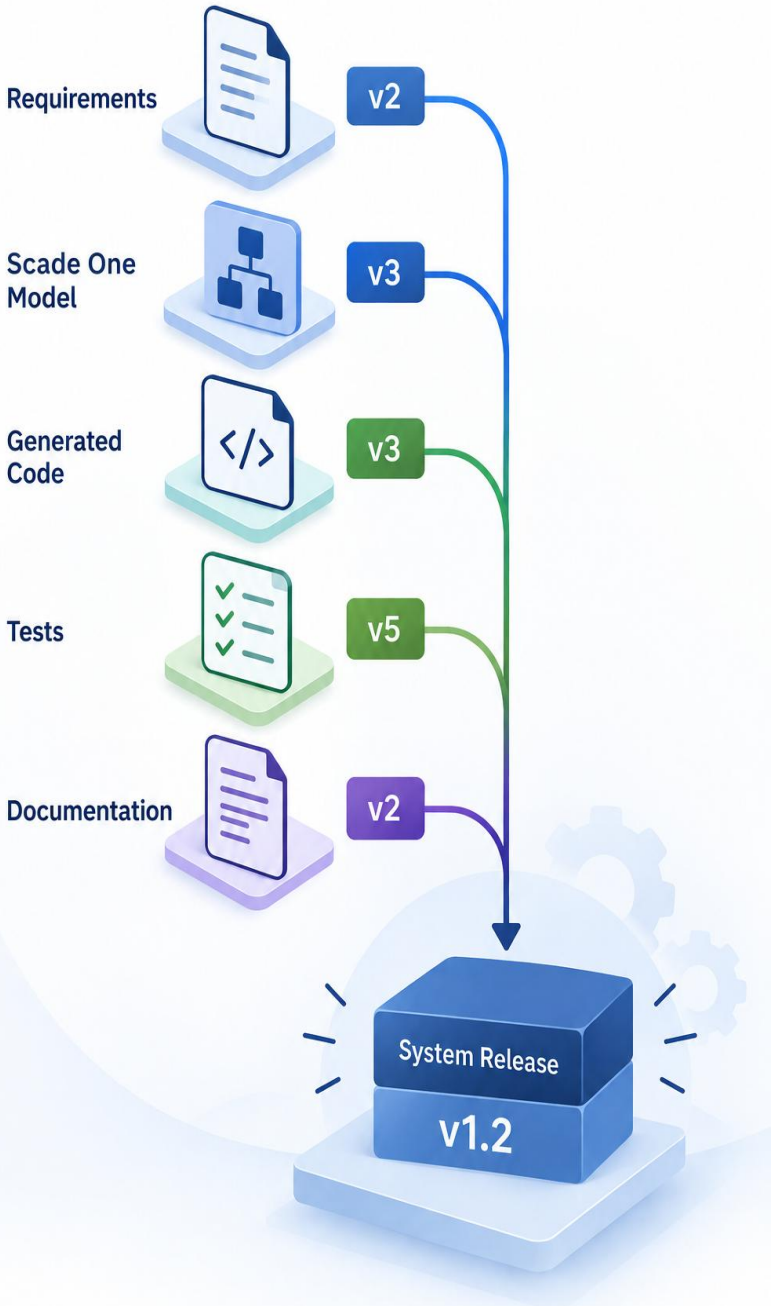






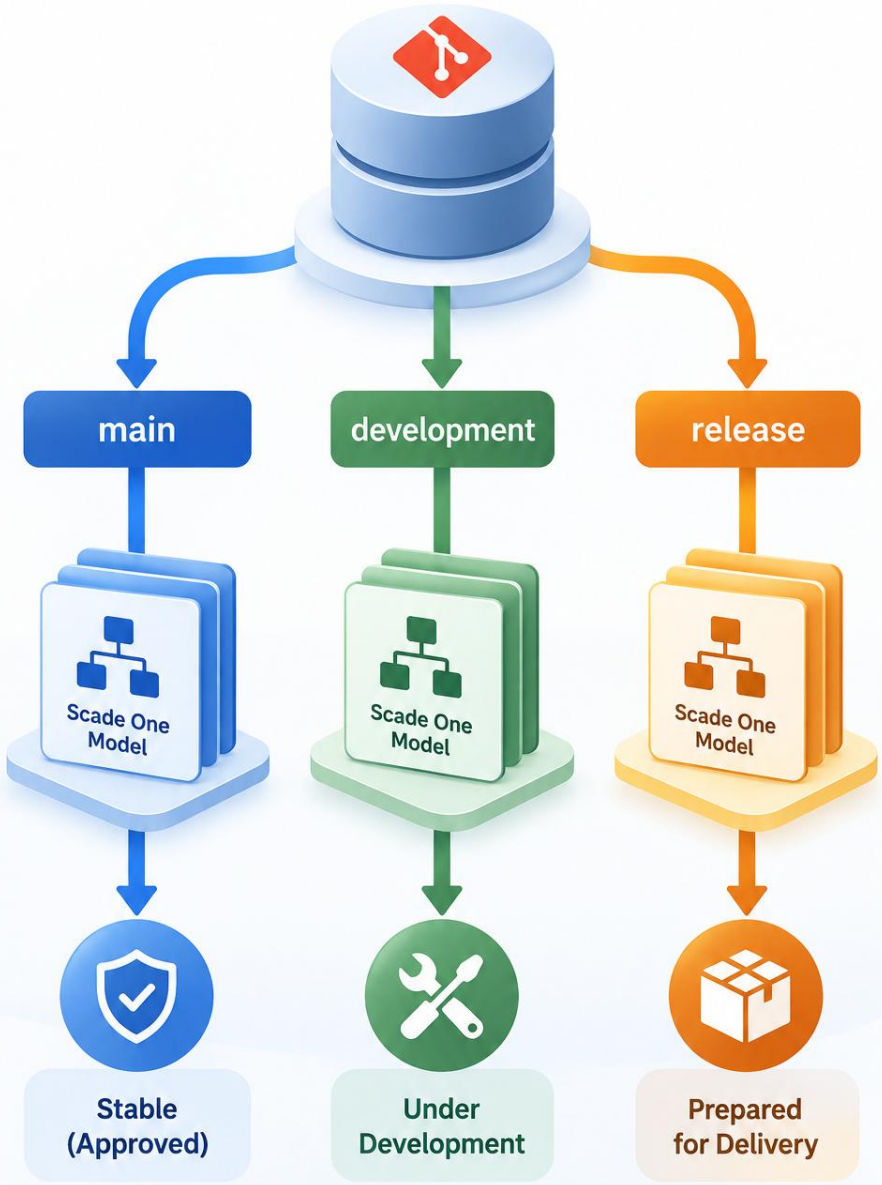
Versioning Strategy

- Versioning is the practice of assigning unique identifiers to software artifacts and their successive states throughout the Software Development Life Cycle (SDLC).
- Main Objective:
  - To ensure that every software artifact can be uniquely identified, retrieved, and associated with the correct system configuration.
- Key Principles:
  - Unique identification: Each version has a distinct identifier (e.g., v1.0, v1.1, v2.0).
  - Consistency: Compatible versions of models, code, tests, and documentation are associated with the same system configuration.
  - Traceability: Every software release can be linked to its exact artifact versions.
- Example – Scade One Project
  - Release v1.2 = SCADE Model v3 + Generated Code v3 + Tests v5 + Requirements v2
- Versioning ensures that we know exactly which artifacts belong to each software release.



Branching Strategy

- Branching is the practice of creating separate, parallel lines of development within a version-control system, such as Git.
- Main Objective
  - To maintain independent and well-organized versions of a software project while preserving the integrity of the main, approved configuration.
- Key Principles
  - Isolation: Different development activities are maintained in separate branches.
  - Stability: The main branch represents a stable, approved project configuration.
  - Traceability: Each branch has its own identifiable history and associated artifacts.
- Example – Scade One Project
  - Main Branch → Approved Scade One model
  - Development Branch → Model under development
  - Release Branch → Configuration prepared for delivery
- Branching allows multiple project configurations to coexist without compromising the approved version.



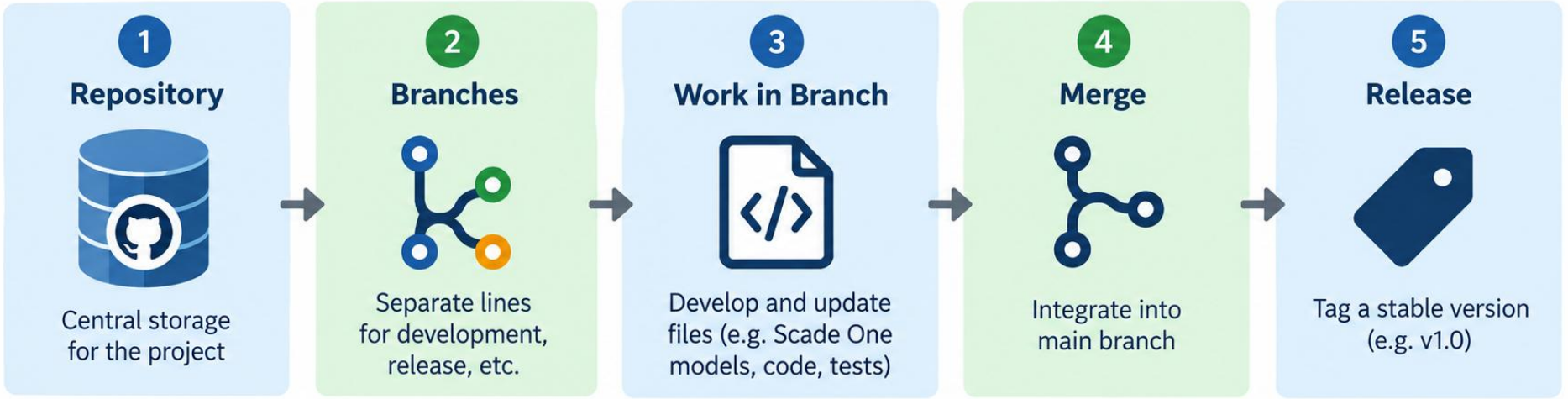
Git as a Configuration Management Tool

- Git provides distributed version control for project artifacts.
- Core concepts:
  - **Repository → Commit → Branch → Merge → Tag/Release**
  - **Repository** – controlled project history
  - **Commit** – identified set of changes
  - **Branch** – isolated development activity
  - **Merge** – integration of approved changes
  - **Tag** – identification of an important baseline/version
- Example:

  main → v1.0 → v1.1 → v2.0
- Each commit creates an identifiable point in the project's history.
- Git helps provide:
  - Change history
  - Version identification
  - Accountability
  - Rollback capability
  - Comparison between versions

GitHub Adds the Control Workflow

- Git provides **version control**; GitHub can add a **collaborative control process.**
- A typical critical-software workflow can be:



Key GitHub Elements

Repository
- Stores all project files
- Complete history

Branches
- Separate development lines
- Keep main branch stable

Files
- Models (Scade One)
- Code, tests, documents
- Versioned automatically

Merge
- Integrate work into main branch
- Maintain a consistent configuration

Tags / Releases
- Identify stable versions
- Support traceability and reproducibility

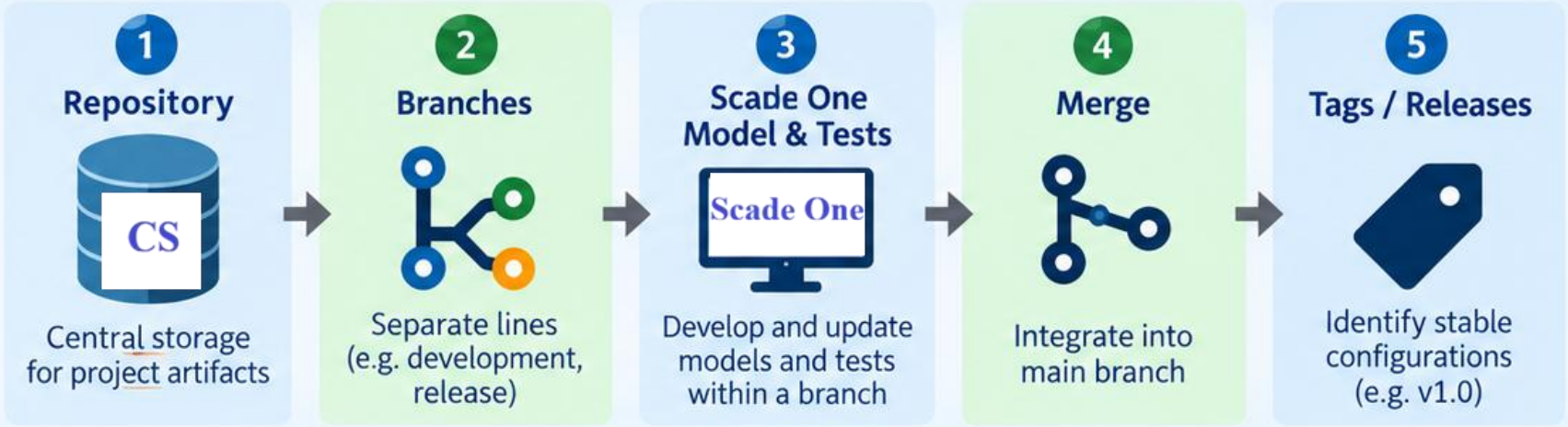
GitHub supports the CM process, but using GitHub alone does **not automatically make a project compliant with a safety standard.**

Configuration Management for a Scade One Project

- Consider a safety-critical controller developed with Scade One.
- The repository organizes and stores all configuration items (CIs) of the project:

```
CriticalController/
├── requirements/
├── scade-model/
├── interfaces/
├── tests/
├── generated-code/
├── verification/
└── documentation/
```

Using GitHub for Configuration Management



Example

REQ-SAFE-021:

"The controller shall enter Safe Mode when sensor communication is lost."

Configuration Management with GitHub

- Create a dedicated branch (e.g. feature/REQ-SAFE-021)
- Update the Scade One model and corresponding tests
- Commit the updated artifacts
- Review and verify the configuration (model + tests + requirements)
- Merge into the main branch
- Tag a release for the approved configuration

Result: a well-organized and traceable set of project configurations.

Traceability Through Configuration Management

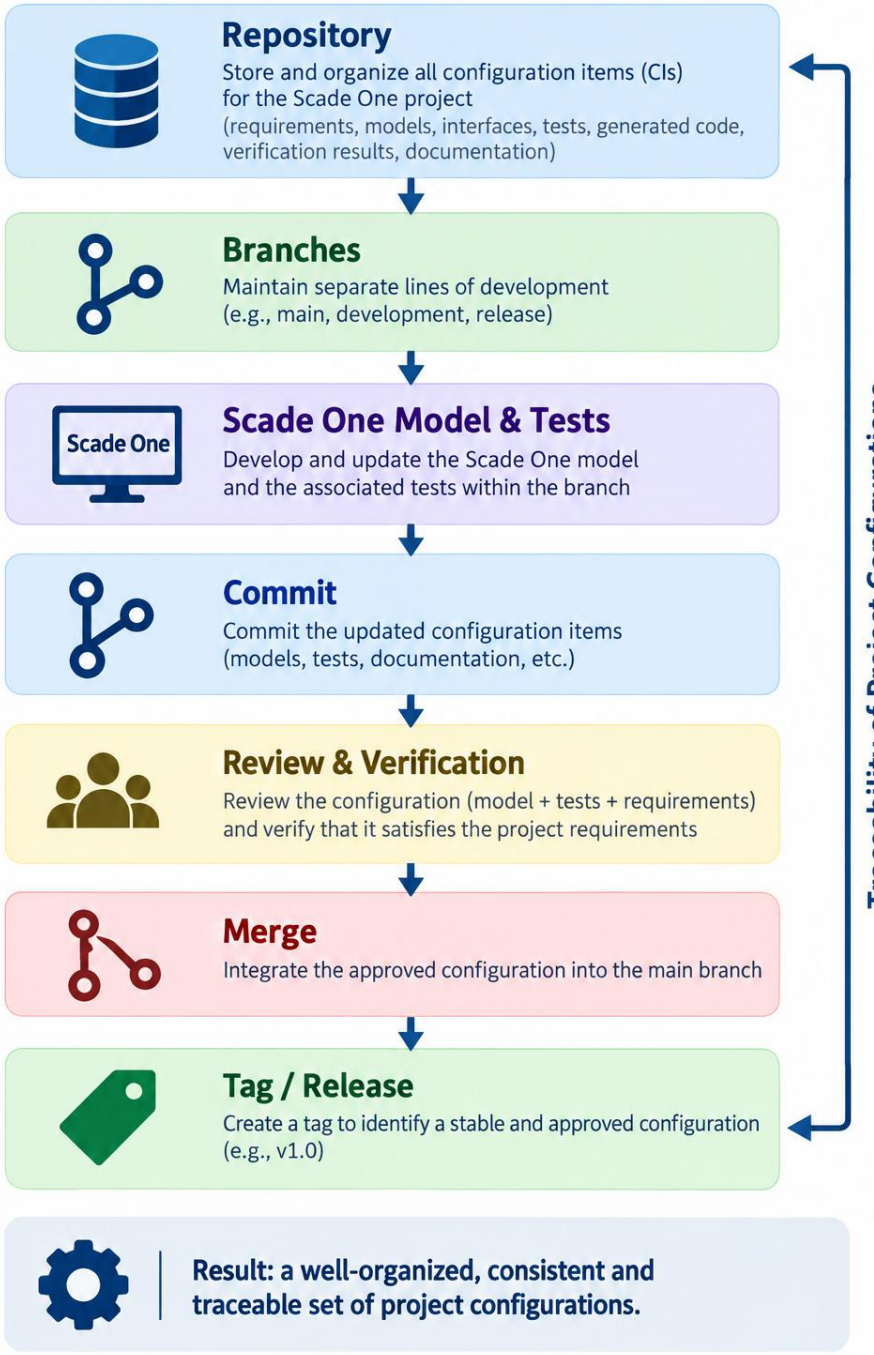
- Configuration management complements requirements traceability.
- A safety-related modification can be connected through:
- Example:

  REQ-SAFE-021 → Issue #47 → Commit a73f... → TEST- 021 → Release v2.1

- This helps answer an important certification question:

  Can we demonstrate exactly how a safety requirement is implemented and verified in this released version?

- This is the connection between configuration management and traceability.



Summary



- 1. Configuration Management is essential for safety
  Safety-critical software requires controlled, identifiable, and approved configurations throughout the lifecycle. CM is therefore an integral part of compliance with safety standards.
- 2. Control more than just source code
  Requirements, Scade One/Swan models, interfaces, tests, generated code, tool configurations, verification results, and documentation should be managed as configuration items when relevant to the released system.
- 3. Git and GitHub enable control
  Git provides versioning and history, while GitHub supports the collaborative workflow around issues, branches, pull requests, reviews, approvals, CI, and releases. Scade One remains the engineering environment where models are developed and verified.
- 4. Traceability + Baselines + Reproducibility = Confidence
  A rigorous CM process connects:

  Hazard → Requirement→ Model → Test → Verification → Release

  Approved baselines make it possible to identify, retrieve, reproduce, audit, and maintain a specific critical-software release.

Configuration Management ensures that we know exactly what was built, what versions we have, how it was verified, and what was finally released.